DEVIXO SOLUTIONS

AI/ML INTERNSHIP PROGRAM

TASK 02

1. Introduction

This project focuses on developing and comparing multiple machine learning classification models for a real-world customer churn prediction problem. The dataset contains information about bank customers, including credit score, geography, gender, age, account balance, number of products, activity status, and estimated salary.

The main objective is to predict whether a customer is likely to leave the bank. Three different machine learning algorithms were developed and evaluated: **Logistic Regression, Decision Tree, and Random Forest**.

The project also includes data preprocessing, categorical encoding, feature scaling, train-test splitting, model evaluation, model comparison, and hyperparameter tuning using GridSearchCV.

---

2. Objective

The objectives of this project are:

- To preprocess a real-world customer dataset.
- To identify and prepare relevant features for machine learning.
- To handle categorical and numerical data appropriately.
- To split the dataset into training and testing sets.
- To train at least three different classification algorithms.
- To evaluate models using Accuracy, Precision, Recall, and F1 Score.
- To analyze confusion matrices for each model.
- To compare the performance of different models.
- To perform hyperparameter tuning using GridSearchCV.
- To select and recommend the best-performing model.

---

3. Technologies Used

- **Python** — Programming language
- **Pandas** — Data loading and manipulation
- **NumPy** — Numerical operations
- **Scikit-learn** — Preprocessing, model training and evaluation
- **Matplotlib** — Data visualization
- **Seaborn** — Statistical visualization

---

4. Dataset Description

The project uses a **Bank Customer Churn** dataset containing **10,000 records**.

Initially, the dataset contained **14 columns**. Three identifier columns — `RowNumber`, `CustomerId`, and `Surname` — were removed because they do not provide meaningful predictive information for customer churn.

After removing these columns, the dataset contained **11 columns**.

The target variable is:

`Exited`

where:

- `0` = Customer did not leave the bank
- `1` = Customer left the bank

Main Features

Feature	Description
CreditScore	Customer's credit score
Geography	Customer's country/region
Gender	Customer gender

Age	Customer age
Tenure	Number of years with the bank
Balance	Account balance
NumOfProducts	Number of bank products used
HasCrCard	Whether customer has a credit card
IsActiveMember	Whether customer is an active member
EstimatedSalary	Estimated customer salary
Exited	Target variable indicating churn

Dataset size: 10,000 rows × 14 original columns.

After feature selection: 10 input features + 1 target = 11 columns.

5. Data Preprocessing

Before training the machine learning models, the dataset was prepared through several preprocessing steps.

5.1 Feature Selection

The columns `RowNumber`, `CustomerId`, and `Surname` were removed because they are identification-related fields and do not provide useful information for predicting customer churn.

The remaining features were divided into:

- **Input features (`X`)** — customer information used for prediction.
- **Target (`y`)** — `Exited`, which indicates whether the customer left the bank.

After feature selection:

10 input features + 1 target = 11 columns

5.2 Missing Value Check

The dataset was checked for missing values using:

df.isnull().sum()

The dataset contained **no missing values**, so no missing-value imputation was required.

5.3 Duplicate Check

Duplicate records were checked using:

df.duplicated().sum()

The result was **0**, meaning no duplicate records were found.

5.4 Categorical Encoding

The dataset contained two categorical features:

- `Geography`
- `Gender`

These categorical values were converted into numerical representations using **One-Hot Encoding**.

OneHotEncoder(handle_unknown="ignore")

Before encoding, the feature matrix contained:

10 features

After encoding:

13 features

This conversion was necessary because machine learning algorithms work with numerical input.

5.5 Feature Scaling

After encoding, numerical features were standardized using `StandardScaler`.

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X_encoded)

Scaling puts features on a comparable numerical scale and is particularly useful for models such as Logistic Regression.

The resulting feature matrix remained:

(10,000, 13)

5.6 Train-Test Split

The processed dataset was divided into training and testing sets using `train_test_split`.

```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

The dataset was divided into:

- **Training data:** 8,000 records
- **Testing data:** 2,000 records

- **Training features:** 8,000 × 13
- **Testing features:** 2,000 × 13

The training data was used to train the machine learning models, while the testing data was kept for evaluating their performance.

5.7 Preprocessing Summary

Step	Result
Original dataset	10,000 × 14
Removed identifier columns	3
Features after selection	10
Missing values	None
Duplicate records	0
Encoded features	13
Training data	8,000
Testing data	2,000

6. Model Development

After completing the preprocessing stage, three different machine learning classification algorithms were trained to predict customer churn. The selected models were **Logistic Regression, Decision Tree, and Random Forest**.

The purpose of using multiple algorithms was to compare their performance and identify the model that provides the best results for the given customer churn prediction problem.

6.1 Logistic Regression

Logistic Regression was used as the baseline classification model. It is a supervised machine learning algorithm commonly used for binary classification problems.

In this project, the model predicts whether a customer will:

- `0` → Stay with the bank
- `1` → Leave the bank

The model was trained using the processed training dataset.

log_model = LogisticRegression(random_state=42)

log_model.fit(X_train, y_train)

The training time for Logistic Regression was approximately **0.02 seconds**.

Logistic Regression was selected because it is simple, fast, and provides a useful baseline for comparing more complex models.

6.2 Decision Tree

The second model used was a **Decision Tree Classifier**.

A Decision Tree makes predictions by dividing the data through a sequence of conditions. Each split separates customers based on their feature values until a final prediction is reached.

```
tree_model = DecisionTreeClassifier(random_state=42)
```

```
tree_model.fit(X_train, y_train)
```

The Decision Tree required approximately **0.04 seconds** for training.

The model is easy to understand and can capture nonlinear relationships between customer characteristics and churn.

However, individual decision trees can become complex and may overfit the training data.

6.3 Random Forest

The third and most advanced model used was the **Random Forest Classifier**.

Random Forest is an ensemble learning algorithm that combines predictions from multiple Decision Trees. In this project, **100 trees** were used.

```
rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)
```

```
rf_model.fit(X_train, y_train)
```

The original Random Forest required approximately **1.44 seconds** for training.

Although it required more training time than Logistic Regression and Decision Tree, Random Forest achieved considerably better overall evaluation results.

6.4 Training Time Comparison

Model	Approx. Training Time
Logistic Regression	0.02 sec
Decision Tree	0.04 sec
Random Forest	1.44 sec

The training times can vary slightly between different executions depending on the system environment.

Model Development Summary

Model	Type	Main Characteristic
Logistic Regression	Linear classifier	Fast and simple baseline
Decision Tree	Tree-based classifier	Easy to interpret and handles nonlinear patterns
Random Forest	Ensemble classifier	Combines multiple decision trees for stronger predictions

7. Model Evaluation

After training the three machine learning models, their performance was evaluated using different classification metrics. Since the project is a **customer churn classification problem**, Accuracy, Precision, Recall, F1 Score, and Confusion Matrix were used.

7.1 Accuracy

Accuracy represents the percentage of total predictions that were correctly classified.

A higher accuracy indicates that the model made more correct predictions overall.

7.2 Precision

Precision measures how many of the customers predicted as churners actually left the bank.

A higher precision means the model produces fewer false-positive churn predictions.

7.3 Recall

Recall measures how many of the customers who actually left the bank were successfully identified by the model.

Recall is particularly important in churn prediction because identifying customers who are likely to leave can help the bank take retention actions.

7.4 F1 Score

F1 Score provides a balance between Precision and Recall.

It is useful when both false positives and false negatives are important.

7.5 Model Evaluation Results

The models produced the following results on the **2,000 testing records**:

Model	Accuracy	Precision	Recall	F1 Score

Logistic Regression	80.80%	58.91%	18.67%	28.36%
Decision Tree	79.40%	49.41%	**51.84%**	50.60%
Random Forest	**86.15%**	**76.42%**	46.19%	**57.58%**

Analysis

Logistic Regression achieved an accuracy of 80.80%, but its Recall and F1 Score were relatively low. This indicates that although it correctly classified many customers overall, it identified comparatively fewer customers who actually churned.

Decision Tree achieved 79.40% accuracy. Its main strength was its **Recall of 51.84%**, the highest among the three models. Therefore, it identified more actual churn cases than the other models.

Random Forest achieved the highest overall performance, with **86.15% Accuracy, 76.42% Precision, and 57.58% F1 Score**. Although its Recall was slightly lower than the Decision Tree, it provided the best overall balance between Precision and Recall.

7.6 Confusion Matrix

A confusion matrix shows the number of correct and incorrect predictions by separating them into:

- **True Negative (TN):** Correctly predicted non-churn customers
- **False Positive (FP):** Non-churn customers incorrectly predicted as churn
- **False Negative (FN):** Churn customers incorrectly predicted as non-churn
- **True Positive (TP):** Correctly predicted churn customers

Logistic Regression

[[1540 53]

 [331 76]]

The model correctly classified **1,540 non-churn customers** and **76 churn customers**.

Decision Tree

[[1377 216]

 [196 211]]

The Decision Tree identified **211 actual churn customers**, giving it the highest Recall among the three models.

Random Forest

[[1535   58]

 [219 188]]

Random Forest correctly classified **1,535 non-churn customers** and **188 churn customers**, while maintaining the highest overall accuracy and precision.

7.7 Confusion Matrix Visualization

Three confusion matrix visualizations were generated to visually compare the predictions of the models.

These visualizations make it easier to understand the distribution of correct and incorrect predictions for each algorithm.

8. Model Comparison

After evaluating all three machine learning models, their results were compared based on Accuracy, Precision, Recall, F1 Score, and training time.

8.1 Comparison Table

Model	Accuracy	Precision	Recall	F1 Score	Training Time

Logistic Regression	80.80%	58.91%	18.67%	28.36%	0.02 sec
Decision Tree	79.40%	49.41%	**51.84%**	50.60%	0.04 sec
Random Forest	**86.15%**	**76.42%**	46.19%	**57.58%**	1.44 sec

The results show that **Random Forest achieved the highest Accuracy, Precision, and F1 Score** among the three models.

8.2 F1 Score Comparison

A bar chart was created to compare the F1 Scores of the three models.

The F1 Score comparison shows that:

- Logistic Regression → **28.36%**
- Decision Tree → **50.60%**
- Random Forest → **57.58%**

Random Forest therefore provided the strongest overall balance between Precision and Recall.

8.3 Advantages and Limitations

Model	Advantages	Limitations
Logistic Regression	Very fast, simple, easy to understand	Lower Recall and F1 Score in this dataset
Decision Tree	Easy to interpret and captures nonlinear relationships	Lower overall Accuracy and can overfit

Random Forest	Highest Accuracy, Precision and F1 Score; more robust than a single tree	Higher training time and less interpretable

---

8.4 Model Selection

Based on the evaluation results, **Random Forest was selected as the best overall model**.

It achieved:

- **Accuracy:** 86.15%
- **Precision:** 76.42%
- **Recall:** 46.19%
- **F1 Score:** 57.58%

Although Decision Tree achieved a slightly higher Recall of **51.84%**, Random Forest achieved a considerably better Precision and the highest F1 Score. Therefore, Random Forest provided the best overall performance for this project.

8.5 Training Time Comparison

Random Forest required more training time because it builds and combines multiple decision trees. However, the additional training cost was reasonable compared with its improvement in predictive performance.

9. Hyperparameter Tuning — Bonus

To improve the Random Forest model, hyperparameter tuning was performed using **GridSearchCV**.

GridSearchCV tests different combinations of model parameters and selects the combination that provides the best cross-validation performance.

9.1 Parameters Tested

The following parameters were evaluated:

- `n_estimators` → number of decision trees
- `max_depth` → maximum depth of each tree
- `min_samples_split` → minimum number of samples required to split a node

The parameter grid used was:

```
param_grid = {
    "n_estimators": [50, 100, 150],
    "max_depth": [None, 10, 20],
    "min_samples_split": [2, 5]
}
```

GridSearchCV was configured with **3-fold cross-validation** and **F1 Score** as the optimization metric.

9.2 Best Parameters

GridSearchCV selected the following parameter combination:

Parameter	Best Value
`n_estimators`	100
`max_depth`	None
`min_samples_split`	5

The best cross-validation F1 Score was:

0.5837 ≈ 58.37%

9.3 Tuned Random Forest Evaluation

The tuned Random Forest was then evaluated on the unseen test dataset.

Metric	Tuned Random Forest
Accuracy	**86.20%**
Precision	**76.95%**
Recall	**45.95%**
F1 Score	**57.54%**

Confusion Matrix

[[1537 56]

 [220 187]]

The tuned model correctly classified 1,537 non-churn customers and 187 churn customers.

9.4 Tuning Analysis

The tuned model achieved a slightly higher **Accuracy of 86.20%** and **Precision of 76.95%** compared with the original Random Forest.

However, its test-set F1 Score was **57.54%**, which was almost the same as the original Random Forest F1 Score of **57.58%**.

Therefore, hyperparameter tuning did not produce a significant improvement on the test set, but it demonstrated how GridSearchCV can systematically search for suitable model parameters.

📸 **Yahan apna Best Parameters + CV F1 screenshot** aur **Tuned Random Forest Results screenshot** rakh dena.

---

10. Final Conclusion

The project successfully developed and compared three machine learning classification models for predicting customer churn.

The preprocessing stage included feature selection, categorical encoding, feature scaling, and train-test splitting. Logistic Regression, Decision Tree, and Random Forest were then trained and evaluated using Accuracy, Precision, Recall, F1 Score, and Confusion Matrix.

Among the three models, **Random Forest performed the best overall**, achieving an Accuracy of **86.15%**, Precision of **76.42%**, and F1 Score of **57.58%**.

The Decision Tree achieved a slightly higher Recall of **51.84%**, which means it identified a greater proportion of actual churn customers. However, Random Forest provided a better overall balance between Precision and Recall.

GridSearchCV was also used as a bonus step to tune the Random Forest hyperparameters. The tuned model achieved **86.20% test accuracy**, although its F1 Score remained almost unchanged.

Therefore, **Random Forest is recommended as the final model for this customer churn prediction problem** because it provided the strongest overall predictive performance among the tested algorithms.